

本小节内容

课时 17 作业

课时 17 作业 1

Description

读取 10 个整型数据 12 63 58 95 41 35 65 0 38 44，然后通过选择排序，堆排序，分别对该组数据进行排序，输出 2 次有序结果，每个数的输出占 3 个空格

完成 OJ 作业的同学，可以购买《跟龙哥学 C 语言编程》，有很多课后习题可以练习，附带答案，或者直接 B 站搜王道论坛，看王道的数据结构，组成原理。

Input

12 63 58 95 41 35 65 0 38 44

Output

输出 2 次有序结果

0 12 35 38 41 44 58 63 65 95

0 12 35 38 41 44 58 63 65 95

这道题考察的是选择排序、堆排序，和我们上课写的代码是类似的，只是需要首先通过 for 循环读取 10 个整型元素，下面直接上代码

答案如下：

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
#include <string.h>
typedef int ElemType;
typedef struct {
    ElemType* elem;
    int TableLen;
}SSTable;

void ST_init(SSTable& ST, ElemType A[], int len)
{
    ST.TableLen = len;
    ST.elem = (ElemType*)malloc(sizeof(ElemType) * (ST.TableLen));
    int i;
    srand(time(NULL));
    for (i = 0; i < ST.TableLen; i++)
    {
        //ST.elem[i] = rand() % 100;//这里是为了方便测试
        ST.elem[i] = A[i];//提交到 OJ 时使用这一句
    }
}

void ST_print(SSTable ST)
{
    for (int i = 0; i < ST.TableLen; i++)
    {
        printf("%3d", ST.elem[i]);
    }
    printf("\n");
}

void swap(ElemType& a, ElemType& b)
{
    ElemType tmp;
    tmp = a;
    a = b;
    b = tmp;
}

void SelectSort(ElemType A[], int n)
{
    int i, j, min;//min 记录最小的元素的下标
    for (i = 0; i < n - 1; i++)//最多可以为 8
    {
        min = i;
        for (j = i + 1; j < n; j++)//j 最多可以为 9
        {
```

```
        if (A[j] < A[min])
            min = j;
    }
    if (min != i)
    {
        swap(A[i], A[min]);
    }
}

//调整子树
void AdjustDown(ElemType A[], int k, int len)
{
    int dad = k;
    int son = 2 * dad + 1; //左孩子下标
    while (son <= len)
    {
        if (son + 1 <= len && A[son] < A[son + 1]) //看下有没有右孩子，比较左右孩子选大的
        {
            son++;
        }
        if (A[son] > A[dad]) //比较孩子和父亲
        {
            swap(A[son], A[dad]);
            dad = son;
            son = 2 * dad + 1;
        }
        else {
            break;
        }
    }
}

void HeapSort(ElemType A[], int len)
{
    int i;
    //建立大顶堆
    for (i = len / 2; i >= 0; i--)
    {
        AdjustDown(A, i, len);
    }
    swap(A[0], A[len]); //交换顶部和数组最后一个元素
    for (i = len - 1; i > 0; i--)
    {
        AdjustDown(A, 0, i); //剩下元素调整为大根堆
    }
}
```

```
        swap(A[0], A[i]);
    }
}

//《王道 C 督学营》课程
//选择排序与堆排序
int main()
{
    SSTable ST;
    ElemType A[10];
    int i;
    for (i = 0; i < 10; i++)
    {
        scanf("%d", &A[i]);
    }
    ST_init(ST, A, 10); //把 A 元素复制给 ST
    SelectSort(ST.elem, 10);
    ST_print(ST);
    ST_init(ST, A, 10); //把 A 元素复制给 ST
    HeapSort(ST.elem, 9);
    ST_print(ST);
    return 0;
}
```

课时 17 作业 2

Description

读取 10 个整型数据 12 63 58 95 41 35 65 0 38 44，然后通过归并排序，对该组数据进行排序，输出有序结果，每个数的输出占 3 个空格

Input

12 63 58 95 41 35 65 0 38 44

Output

0 12 35 38 41 44 58 63 65 95

这道题考察的是归并排序，代码和上课编写的一致，直接使用即可，代码如下：

```
#include <stdio.h>
```

```
#define N 10
```

```
typedef int ElemType;
```

```
//49,38,65,97,76,13,27
```

```
void Merge(ElemType A[], int low, int mid, int high)
```

```
{
```

```
    static ElemType B[N]; //为了降低操作次数
```

```
    int i, j, k;
```

```
    for (k = low; k <= high; k++) //复制元素到 B 中
```

```
        B[k] = A[k];
```

```
    for (i = low, j = mid + 1, k = i; i <= mid && j <= high; k++) //合并两个有序数组
```

```
    {
```

```
        if (B[i] <= B[j])
```

```
            A[k] = B[i++];
```

```
        else
```

```
            A[k] = B[j++];
```

```
    }
```

```
    while (i <= mid) //如果有剩余元素，接着放入即可
```

```
        A[k++] = B[i++];
```

```
    while (j <= high)
```

```
        A[k++] = B[j++];
```

```
}
```

```
void MergeSort(ElemType A[], int low, int high) //递归分割
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        int mid = (low + high) / 2;
```

```
        MergeSort(A, low, mid);
```

```
        MergeSort(A, mid + 1, high);
```

```
        Merge(A, low, mid, high);
```

```
    }
```

```
}
```

```
void print(int* a)
```

```
{
```

```
    for (int i = 0; i < N; i++)
```

```
    {
```

```
        printf("%3d", a[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main()
```

```
{
```

```
int A[10];  
int i;  
for (i = 0; i < 10; i++)  
{  
    scanf("%d", &A[i]);  
}  
MergeSort(A, 0, 9);  
print(A);  
return 0;  
}
```